Actividad Integradora 3

TiendaTec: Integración de una API REST Pública y Gestión de Estados en una Aplicación Móvil Flutter

Alumno: Lenin Montenegro Santana

Materia: Desarrollo de Aplicaciones Móviles

Profesor: Gregorio Cruz Gonzabay

8 de noviembre de 2026

Índice

Sección I: Selección y Descripción de la API Pública 1

API Seleccionada 1

Justificación de la Elección 2

Estructura del JSON Consumido 2

Sección II: Arquitectura y Manejo de Estados 3

Gestión de Estados con Provider 5

Sección III: Extractos de Código Clave 5

Consumo HTTP y Manejo de Errores de Red 5

Mapeo del Modelo JSON 6

Manejo de Estados en el Provider 7

Manejo de Estados en la Interfaz 7

Sección IV: Evidencias de Pruebas (QA) y Enlace al Repositorio 8

Enlace al Repositorio 8

Los Tres Estados de la Petición 9

Interacción y Actualización Manual 10

Evidencia de Control de Versiones 11

Sección V: Conclusiones 13

Importancia del Manejo Asíncrono 13

Importancia de la Gestión de Errores de Red 13

Lecciones del Proceso 14

Uso de Herramientas de Inteligencia Artificial 14

TiendaTec: Integración de una API REST Pública y Gestión de Estados en una Aplicación Móvil Flutter

> **Nota:** este proyecto **continúa y mejora TiendaTec, la aplicación desarrollada en las Actividades Integradoras 1 y 2** (repositorio appmovil_integradora1). En esta tercera fase el catálogo deja de depender de un archivo JSON propio y pasa a consumirse en tiempo real desde una API REST pública, con manejo explícito de los estados de la petición. Las entregas anteriores se conservan en los tags v1.0-integradora1 y v2.0.0.

Sección I: Selección y Descripción de la API Pública

API Seleccionada

Se eligió **DummyJSON**, una API REST pública y gratuita que ofrece datos de prueba realistas en formato JSON. Su recurso *products* no requiere token, registro ni autenticación, tal como exige la actividad, y se documenta en https://dummyjson.com/docs/products.

Tabla 1

Datos Generales de la API Consumida

Elemento	Detalle
Nombre	DummyJSON – Products
Tipo	API REST pública, gratuita, sin token ni sesión
Método	GET (petición asíncrona con el paquete http)
Endpoint	https://dummyjson.com/products/category/{categoria}
Categorías usadas	smartphones, laptops, tablets y mobile-accessories
Parámetro	select=title,description,price,thumbnail,category,rating
Formato	JSON (objeto con la lista products)

Justificación de la Elección

TiendaTec es una tienda de tecnología, por lo que se necesitaba una fuente con productos reales: nombre, descripción, precio, imagen, categoría y calificación. DummyJSON entrega exactamente esos campos y permite filtrar por categoría, de modo que la app muestra solo celulares, laptops, tablets y accesorios en lugar de productos de belleza o alimentos que también contiene la API. Además, el parámetro select reduce la respuesta a los campos que la aplicación usa, lo que disminuye los datos transferidos en redes móviles.

Se descartaron otras opciones: Fake Store API presenta caídas frecuentes, y APIs como OpenWeather exigen una clave de acceso. Con DummyJSON la funcionalidad se integra en la aplicación existente sin cambiar su propósito.

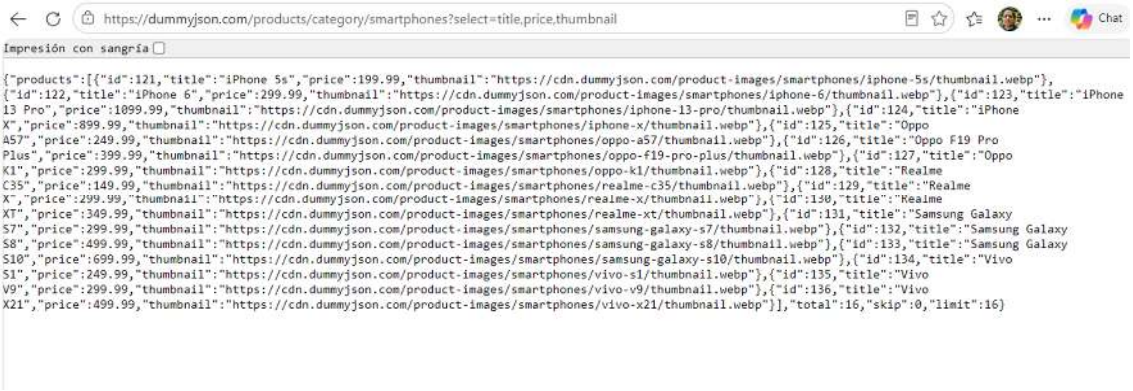
Estructura del JSON Consumido

Cada categoría se solicita con una URL como https://dummyjson.com/products/category/smartphones?select=title,description,price,thumbnail,category,rating. La respuesta es un objeto cuya clave products contiene la lista de productos; las claves total, skip y limit sirven para paginación:

```
{
  "products": [
    {
      "id": 0,              // entero: identificador del producto
      "title": "...",       // texto: nombre
      "description": "...",  // texto: descripción
      "category": "...",     // texto: categoría
      "price": 0.0,          // número: precio en dólares
      "rating": 0.0,         // número: calificación de 0 a 5
      "thumbnail": "https://..."  // texto: URL de la imagen
    }
  ],
  "total": 0,  "skip": 0,  "limit": 0
}
```

Figura 1

Respuesta Real del Endpoint de Smartphones Vista en el Navegador



Nota. Captura propia del navegador. Se observa la lista products con los campos solicitados mediante select y, al final, total, skip y limit.

Tabla 2

Mapeo de las Claves del JSON al Modelo Producto

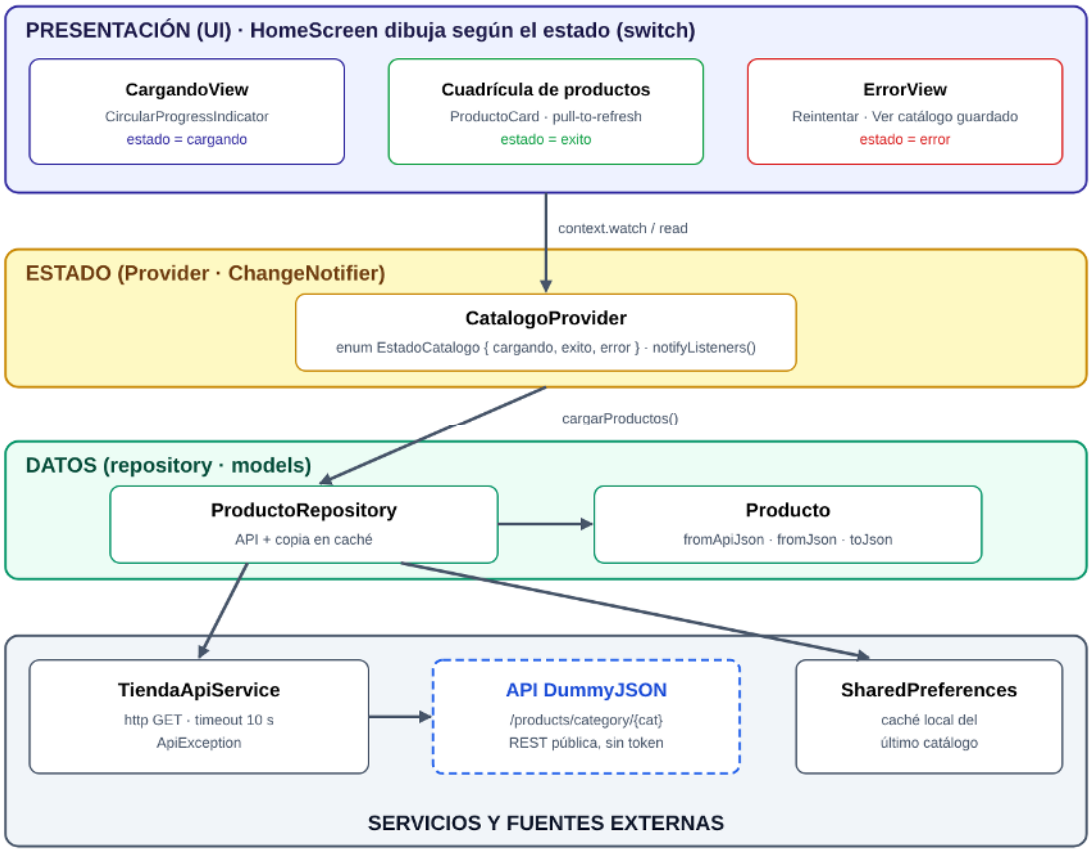
Clave JSON	Tipo	Atributo Dart	Valor por defecto si llega null
id	num	id (int)	0
title	String	nombre	“Producto sin nombre”
description	String	descripcion	“Sin descripción”
price	num	precio (double)	0
thumbnail	String	imagen	cadena vacía (se muestra un ícono)
category	String	categoria	“General”
rating	num	calificacion (double)	0

Sección II: Arquitectura y Manejo de Estados

La lógica de red está desacoplada de la interfaz mediante capas. Las pantallas no importan el paquete http ni conocen la URL de la API: solo observan el estado del CatalogoProvider y dibujan la vista que corresponde (Figura 2).

Figura 2

Arquitectura de TiendaTec Para el Consumo de la API



Nota. Elaboración propia. Las flechas indican dependencia: la UI depende del Provider, el Provider del repositorio y solo el servicio realiza peticiones HTTP.

Tabla 3

Responsabilidad de Cada Capa

Capa	Clase	Responsabilidad

Capa	Clase	Responsabilidad
Servicio	TiendaApiService	Construye la URL, ejecuta el GET, valida el código HTTP, convierte el JSON y transforma los errores en ApiException.
Modelo	Producto	fromApiJson (claves de la API), fromJson y toJson (formato de la caché).
Repositorio	ProductoRepository	Pide los datos al servicio, guarda una copia en SharedPreferences y la entrega cuando no hay red.
Estado	CatalogoProvider	Mantiene EstadoCatalogo (cargando, éxito, error) y avisa a la UI con notifyListeners().
Presentación	HomeScreen y widgets	CargandoView, cuadrícula con ProductoCard y ErrorView, según el estado.

Gestión de Estados con Provider

Se mantuvo Provider, el patrón ya usado en las fases anteriores y en clase. El CatalogoProvider extiende ChangeNotifier y expone un enum EstadoCatalogo { cargando, exito, error }. Al iniciar la petición el estado pasa a **cargando**; si la API responde correctamente pasa a **éxito** con la lista de productos; si ocurre cualquier falla de red pasa a **error** con un mensaje comprensible para el usuario. Cada cambio llama a notifyListeners() y la pantalla, que escucha con context.watch, se redibuja sola. Los botones usan context.read porque solo ejecutan una acción y no necesitan redibujarse.

Esta separación permite cambiar la API o la forma de guardar la caché sin modificar ninguna pantalla, y facilita probar cada capa por separado.

Sección III: Extractos de Código Clave

Consumo HTTP y Manejo de Errores de Red

```
Future<List<Producto>> obtenerProductos() async {
  final listas = await Future.wait(categorias.map(_obtenerCategoria));
  final productos = listas.expand((lista) => lista).toList();
  debugPrint('API: ${productos.length} productos recibidos de $_host');
  return productos;
}

Future<List<Producto>> _obtenerCategoria(String categoria) async {
  try {
    final respuesta = await _cliente
        .get(urlCategoria(categoria))
        .timeout(const Duration(seconds: 10));

    if (respuesta.statusCode != 200) {
      throw ApiException(
        'El servidor respondió con error ${respuesta.statusCode}.',
      );
    }

    // La respuesta es un objeto: { "products": [...], "total": 16, ... }
    final datos =
        jsonDecode(utf8.decode(respuesta.bodyBytes)) as Map<String, dynamic>;
    final lista = datos['products'] as List<dynamic>? ?? [];

    return lista
        .map((e) => Producto.fromApiJson(e as Map<String, dynamic>))
        .toList();
  } on SocketException {
    throw const ApiException(
      'Sin conexión a internet. Revisa tu red e inténtalo de nuevo.',
    );
  } on TimeoutException {
    throw const ApiException(
      'El servidor tardó demasiado en responder. Inténtalo de nuevo.',
    );
  } on http.ClientException {
    throw const ApiException(
      'No se pudo conectar con el servidor de productos.',
    );
  } on FormatException {
    throw const ApiException('La respuesta del servidor no es válida.');
  } on TypeError {
    throw const ApiException('La respuesta del servidor no es válida.');
  }
}
```

obtenerProductos lanza las cuatro peticiones de categoría al mismo tiempo con Future.wait y une los resultados con expand; así el tiempo total es el de la petición más

lenta y no la suma de las cuatro. Cada GET tiene un timeout de 10 segundos y valida que el statusCode sea 200. Las excepciones técnicas (SocketException, TimeoutException, ClientException, FormatException) se capturan con on y se convierten en una ApiException con un mensaje claro, de modo que la interfaz nunca recibe un error crudo.

Mapeo del Modelo JSON

```
factory Producto.fromApiJson(Map<String, dynamic> json) {
  return Producto(
    id: (json['id'] as num?)?.toInt() ?? 0,
    nombre: json['title'] as String? ?? 'Producto sin nombre',
    descripcion: json['description'] as String? ?? 'Sin descripción',
    precio: (json['price'] as num?)?.toDouble() ?? 0,
    imagen: json['thumbnail'] as String? ?? '',
    categoria: json['category'] as String? ?? 'General',
    calificacion: (json['rating'] as num?)?.toDouble() ?? 0,
  );
}
```

El constructor factory Producto.fromApiJson traduce las claves en inglés de la API al modelo en español de TiendaTec. Aplica programación defensiva: cada campo se lee como tipo anulable (as String?, as num?) y el operador ?? asigna un valor seguro si llega null. Se usa num para el precio y la calificación porque la API puede enviar enteros o decimales, y toDouble() los normaliza. El método toJson permite guardar los productos en la caché.

Manejo de Estados en el Provider

```
Future<void> cargarProductos({bool mostrarCargando = true}) async {
  if (mostrarCargando) {
    _estado = EstadoCatalogo.cargando;
    notifyListeners();
  }

  try {
    _productos = await _repositorio.obtenerProductos();
    _origen = OrigenDatos.red;
    _guardadoEl = null;
    _estado = EstadoCatalogo.exito;
  } on ApiException catch (e) {
    _mensajeError = e.mensaje;
    _respaldo = await _repositorio.leerCache();
    _estado = EstadoCatalogo.error;
    debugPrint('CatalogoProvider: ${e.mensaje}');
  } catch (e) {
    _mensajeError = 'Ocurrió un error inesperado al cargar el catálogo.';
    _respaldo = await _repositorio.leerCache();
```

```
      _estado = EstadoCatalogo.error;
      debugPrint('CatalogoProvider: $e');
    }
    notifyListeners();
  }
```

El método recorre los tres estados. El parámetro mostrarCargando es false en el pull-to-refresh, porque el RefreshIndicator ya muestra su propio indicador y no tiene sentido ocultar los productos mientras se actualizan. Cuando ocurre un error, además del mensaje, se lee la última copia guardada para ofrecer el botón “Ver catálogo guardado”.

Manejo de Estados en la Interfaz

```
body: Column(
  children: [
    // Banner que aparece solo cuando no hay internet
    const OfflineBanner(),
    Expanded(
      // Cada estado de la petición dibuja una pantalla distinta
      child: switch (catalogo.estado) {
        EstadoCatalogo.cargando => const CargandoView(),
        EstadoCatalogo.error => RefreshIndicator(
          onRefresh: () => context
              .read<CatalogoProvider>()
              .cargarProductos(mostrarCargando: false),
          child: ErrorView(
            mensaje: catalogo.mensajeError,
            onReintentar: () =>
                context.read<CatalogoProvider>().cargarProductos(),
            onVerGuardado: catalogo.hayRespaldo
                ? () => context
                      .read<CatalogoProvider>()
                      .usarCatalogoGuardado()
                : null,
          ),
        ),
        EstadoCatalogo.exito => _CatalogoExito(catalogo: catalogo),
      },
    ),
  ],
```

La pantalla usa una expresión switch sobre el estado: CargandoView muestra un CircularProgressIndicator; el estado de éxito dibuja una cuadrícula (SliverGrid) de tarjetas con imagen descargada de internet, precio y calificación; y ErrorView presenta el mensaje con los botones “Reintentar” y “Ver catálogo guardado”. Tanto la cuadrícula como la vista de

error están dentro de un RefreshIndicator con AlwaysScrollableScrollPhysics, por lo que la actualización manual funciona incluso cuando no hay productos que desplazar.

Sección IV: Evidencias de Pruebas (QA) y Enlace al Repositorio

Enlace al Repositorio

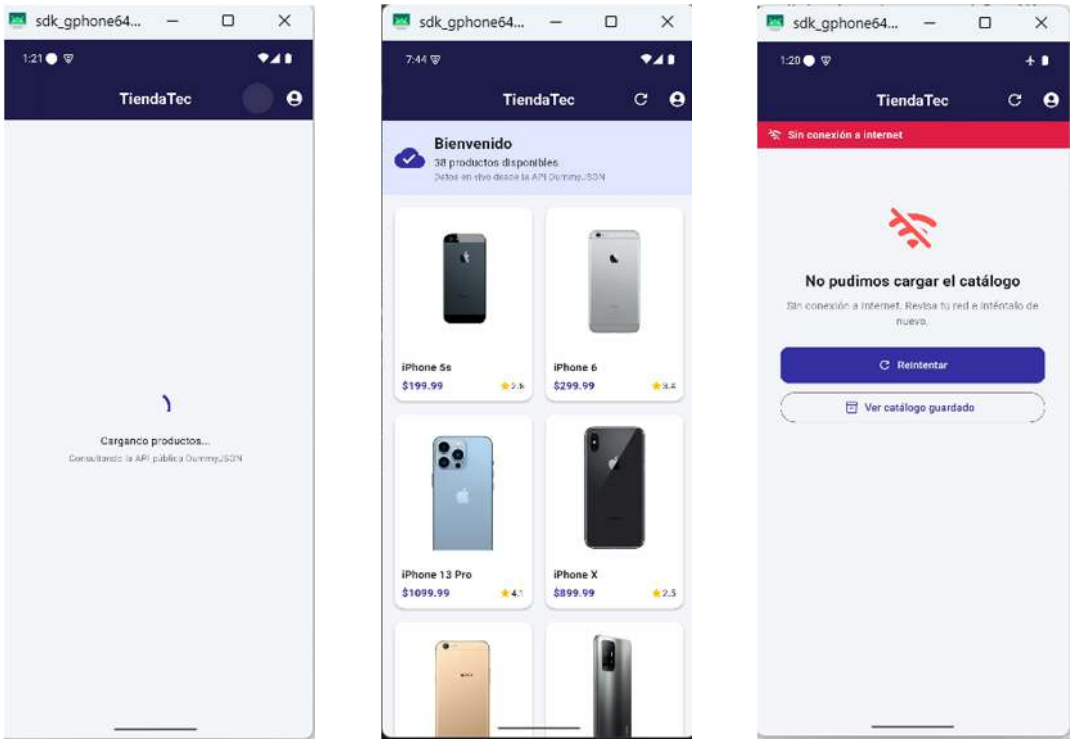
https://github.com/lmontenegro-ecotec/appmovil_integradora1

El repositorio es público. La funcionalidad se desarrolló en la rama feature/api-integration y se integró mediante Pull Requests hacia develop y main. Las pruebas se realizaron en un emulador Pixel 6 con Android 15 (API 35).

Los Tres Estados de la Petición

Figura 3

Estados de la Aplicación al Consumir la API



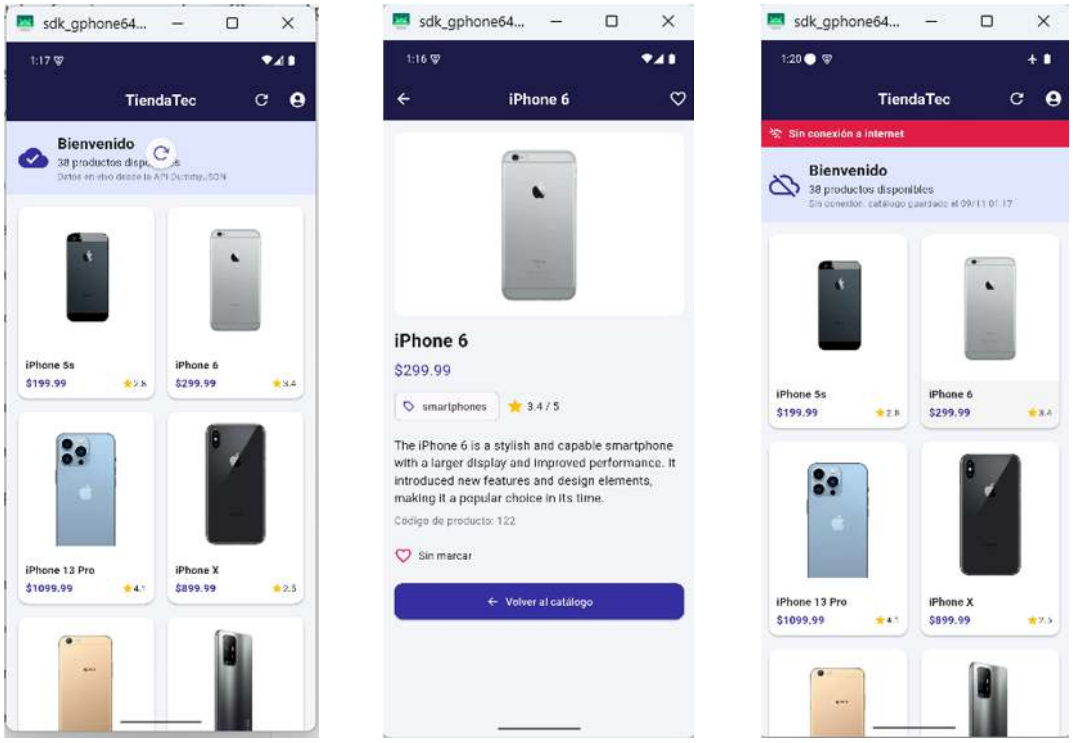
(a) Cargando (b) Éxito: datos de la API (c) Error de conexión

Nota. En (b) el encabezado indica “Datos en vivo desde la API DummyJSON” y 38 productos de tecnología; en (c) se activó el modo avión y se pulsó actualizar.

Interacción y Actualización Manual

Figura 4

Actualización Manual, Detalle de Producto y Respaldo Sin Conexión

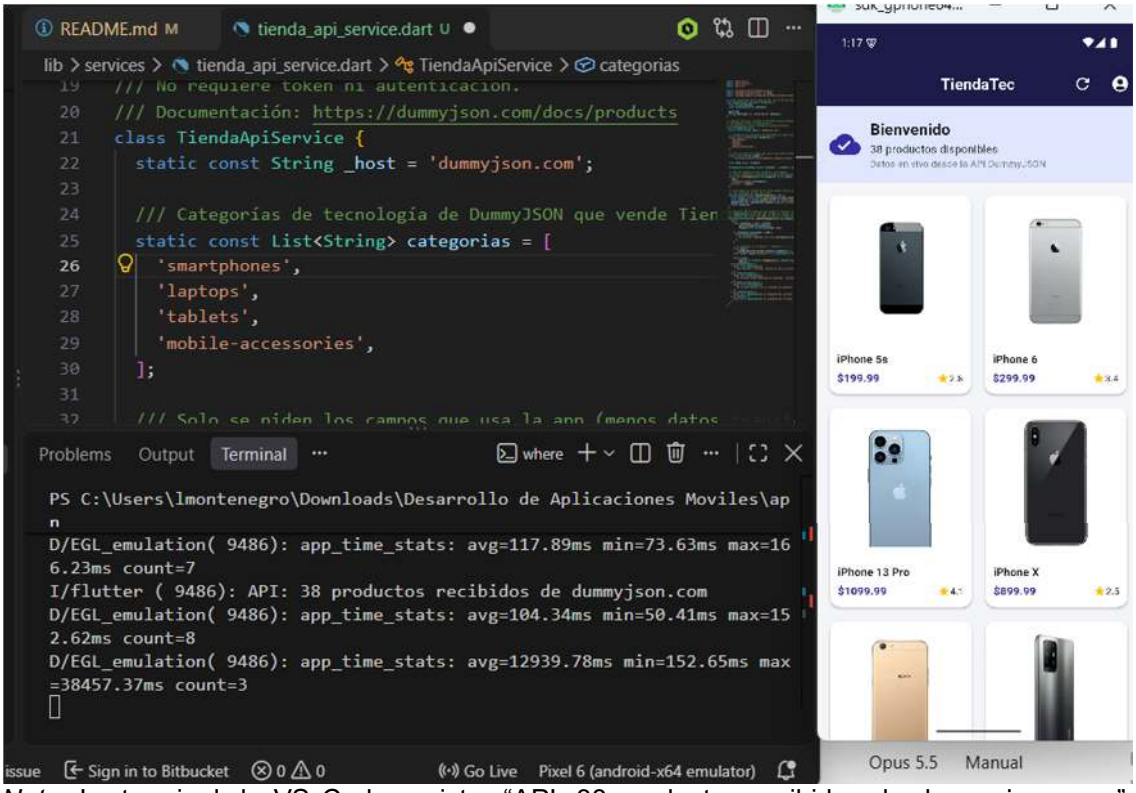


(a) Pull-to-refresh (b) Detalle desde la API (c) Catálogo guardado sin red

Nota. En (a) se observa el indicador del RefreshIndicator al deslizar; en (b) la imagen, la categoría y la calificación recibidas de la API; en (c) el usuario eligió “Ver catálogo guardado” y el encabezado indica la fecha de la copia.

Figura 5

Registro de la Consola al Recibir los Productos



Nota. La terminal de VS Code registra “API: 38 productos recibidos de dummyjson.com”, junto al emulador con el catálogo cargado.

Evidencia de Control de Versiones

Tabla 4

Commits de la Actividad Integradora 3

Commit	Mensaje
0c74cc2	feat(api): servicio TiendaApiService y mapeo del JSON de DummyJSON
5cab08c	feat(estado): estados Cargando, Éxito y Error en CatalogoProvider
badc874	feat(ui): cuadrícula de productos, vistas de carga y error con pull-to-refresh
0b6aa04	docs: README actualizado con la API consumida y cómo probar los estados

Commit	Mensaje
1f1cc6f	Merge pull request #7 (feature/api-integration → develop)
0dc7af9	Merge pull request #8 (develop → main)

Nota. Los cuatro primeros commits se realizaron en la rama feature/api-integration; los dos últimos son los commits de fusión de los Pull Requests.

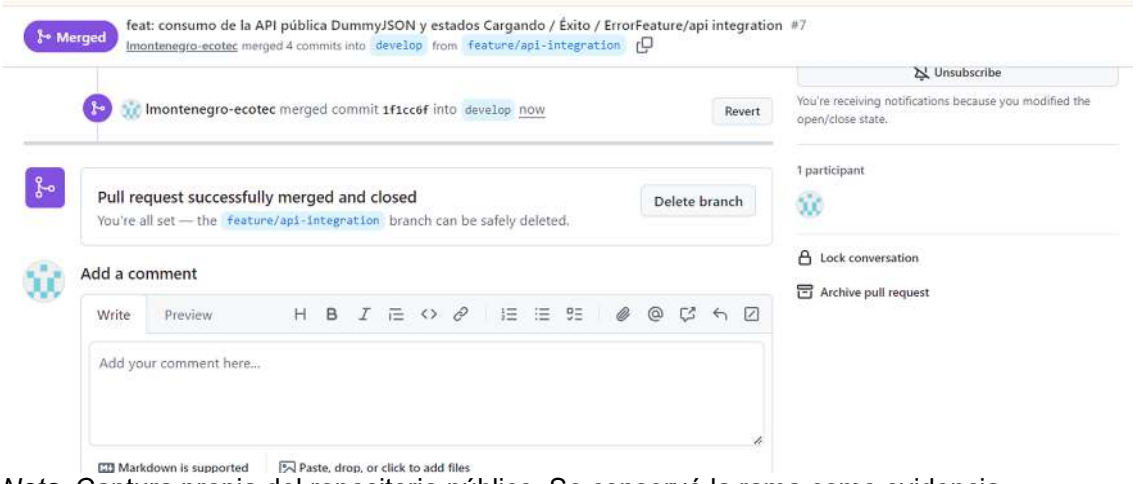
Tabla 5

Pull Requests de la Actividad Integradora 3

PR	Rama	Contenido
#7	feature/api-integration → develop	Consumo de la API DummyJSON y estados Cargando / Éxito / Error (4 commits)
#8	develop → main	Release v3.0.0 de TiendaTec; versión marcada con el tag v3.0.0

Figura 6

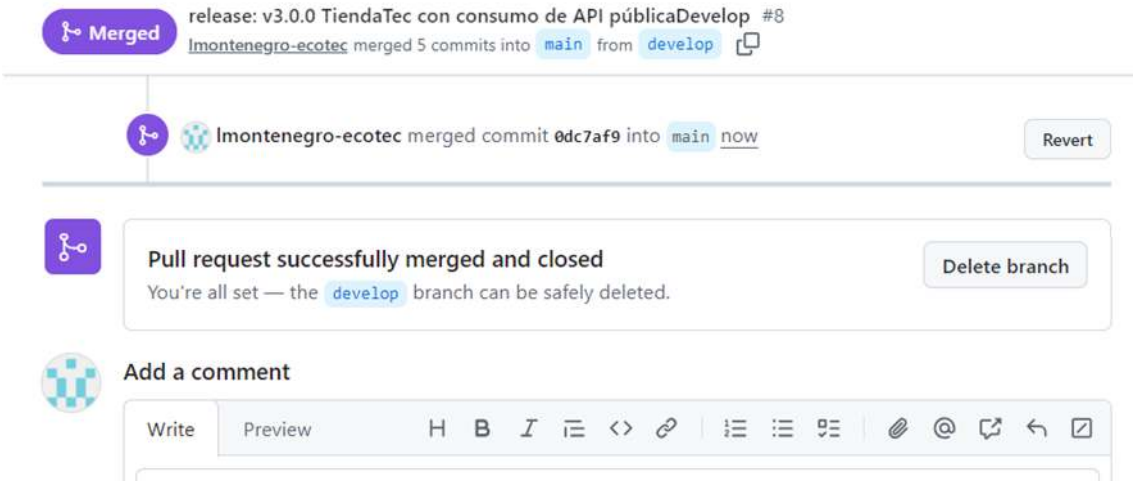
Pull Request #7 Fusionado de feature/api-integration en develop



Nota. Captura propia del repositorio público. Se conservó la rama como evidencia.

Figura 7

Pull Request #8 de develop a main (Release v3.0.0)



Nota. Captura propia del repositorio público.

Sección V: Conclusiones

Importancia del Manejo Asíncrono

Una petición HTTP puede tardar desde milisegundos hasta varios segundos según la red del usuario. Gracias a async, await y Future, la aplicación sigue respondiendo mientras espera: muestra un indicador de carga en lugar de congelarse. También comprobé que la asincronía permite optimizar: con Future.wait las cuatro categorías se descargan en paralelo y el usuario espera el tiempo de la más lenta, no la suma de todas.

Importancia de la Gestión de Errores de Red

En un dispositivo móvil perder la conexión es lo normal, no la excepción: el usuario entra a un ascensor, cambia de Wi-Fi a datos o activa el modo avión. Por eso el estado de error es tan importante como el de éxito. Los timeouts evitan esperas infinitas, la validación del código HTTP detecta fallas del servidor y el mapeo defensivo con tipos anulables y el operador ?? impide que un campo null en la respuesta cierre la aplicación. Convertir las excepciones técnicas en mensajes claros, con opción de reintentar o de ver el último catálogo guardado, convierte una falla en una experiencia controlada.

Lecciones del Proceso

Al probar la primera versión, la API devolvía productos de belleza, que no correspondían a una tienda de tecnología; la solución fue consultar por categorías, lo que además demostró el valor de revisar los datos reales y no solo que la petición funcione. También volví a encontrar que guardar en VS Code un archivo abierto podía sobrescribir una versión corregida, por lo que verifiqué el contenido antes de cada commit. Separar la lógica de red en un servicio y el estado en un Provider facilitó estos cambios: ninguna pantalla tuvo que modificarse para cambiar el endpoint.

Uso de Herramientas de Inteligencia Artificial

Declaro que utilicé Claude (Anthropic) como asistente para seleccionar la API, diagnosticar problemas durante las pruebas y organizar la redacción de este informe a partir de mis capturas y mi código.